

JavaScript Practical Assignment

Part 1 — Syntax Warm-Up

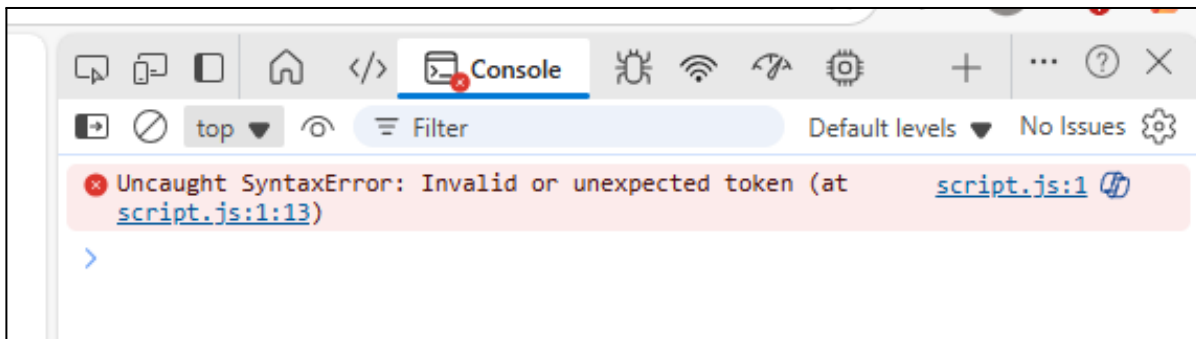
Predicted vs. actual output

Code	Predicted	Actual
<code>console.log("My name is Student Name");</code>	My name is Student Name	My name is Student Name
<code>console.log(5 + 5);</code>	10	10
<code>// comment</code>	No output	No output
<code>console.log("Comments don't run this part");</code>	Comments don't run this part	Comments don't run this part

Task 1.1 — Deliberate syntax error

```
console.log("Hello, JavaScript!)
```

Observation: removing the closing quote breaks the string literal and produces a `SyntaxError`.



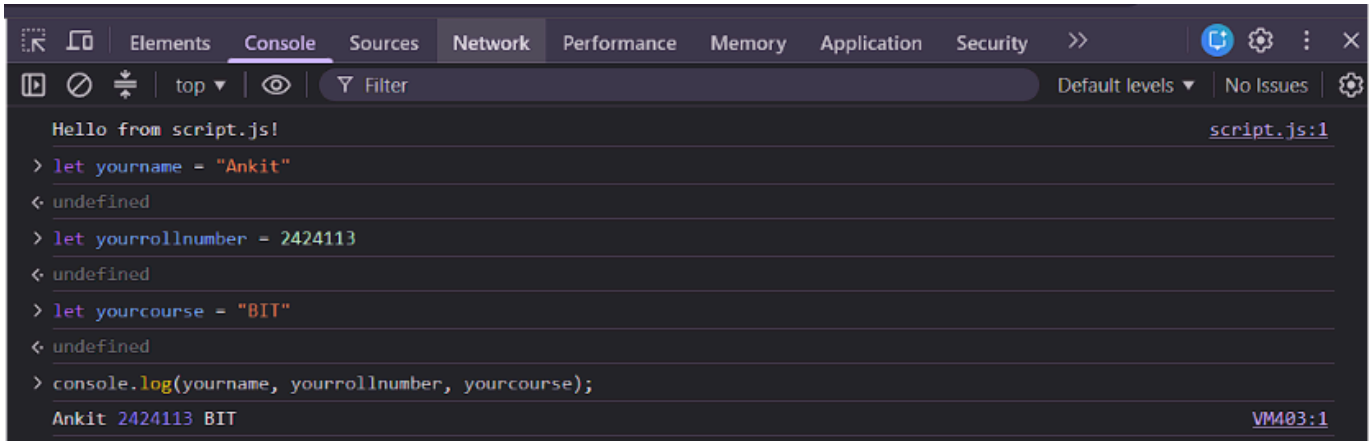
Correction: `console.log("Hello, JavaScript!");`; add an end quote at the end of the string.

Task 1.2 — Case sensitivity

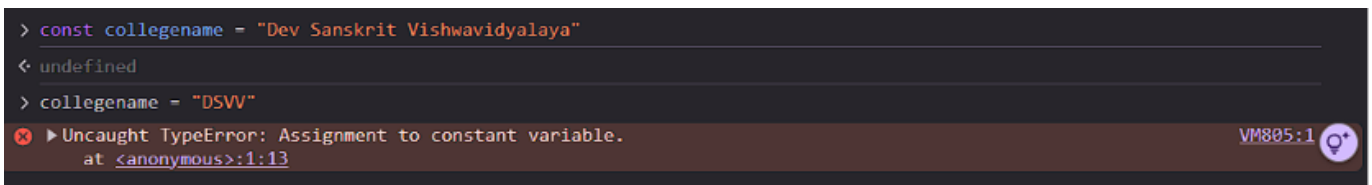
JavaScript is case-sensitive, so a variable named `test` is different from `Test`. It reads capital and small letters as unique symbols.

Part 2 — Variables

Task 2.1 — Declare three `let` variables: your name, your roll number, and your course (BCA or BIT). Print all three in a single `console.log`.

A screenshot of a web browser's developer console. The 'Console' tab is active, showing a series of JavaScript commands and their outputs. The commands are: 'Hello from script.js!', '> let yourname = "Ankit"', '< undefined', '> let yourrollnumber = 2424113', '< undefined', '> let yourcourse = "BIT"', '< undefined', and '> console.log(yourname, yourrollnumber, yourcourse);'. The final output is 'Ankit 2424113 BIT'. The console also shows the source file 'script.js:1' and the VM identifier 'VM403:1'.

Task 2.2 — Try reassigning collegeName to a different value. Write down the exact error message you get.

A screenshot of a web browser's developer console showing a runtime error. The console displays the following code: 'const collegename = "Dev Sanskrit Vishwavidyalaya"', '< undefined', and 'collegename = "DSVV"'. Below the code, an error message is shown: 'Uncaught TypeError: Assignment to constant variable.' at '<anonymous>:1:13'. The console also shows the VM identifier 'VM805:1'.

Task 2.3 — Naming Practice: For each of the following, write “valid” or “invalid” and explain why:
let 1stYear; let student_name; let let; let \$marks; let roll no;

Declaration	valid/Invalid	Reason
let 1stYear;	Invalid	Identifiers cannot start with a digit.
let student_name;	Valid	Underscore is allowed.
let let;	Invalid	Underscore is allowed.
let\$marks;	Valid	\$ is allowed in identifiers.
let roll_no;	Invalid	Spaces are not allowed in identifiers.

Task 2.4 — Scope Challenge: Run the code below. One line works, one throws an error. In your lab record, explain why, in your own words.

```
> if (true){
  var a = "I am var";
  let b = "I am let";
}
< undefined
> console.log(a);
I am var VM971:1
< undefined
> console.log(b);
✖ ▶ Uncaught ReferenceError: b is not defined VM1027:1
   at <anonymous>:1:13
```

Actual observation: console.log(a) prints "I am var". console.log(b) throws a ReferenceError because let is block-scoped, while var is function/global scoped in this context.

Part 3 — Data Types

Task 3.1 — Create one variable for each of the 7 primitive types (String, Number, Boolean, Undefined, Null, BigInt, Symbol) and print each one together with its typeof result.

```
> let str = "Ankit";
< undefined
> let num = 21;
< undefined
> let bool = true;
< undefined
> let undef;
< undefined
> let nul = null;
< undefined
> let bigInt = 5784678465846n;
< undefined
> let sym = Symbol();
< undefined
> console.log(str, typeof str);
Ankit string VM1498:1
< undefined
> console.log(num, typeof num);
21 'number' VM1598:1
< undefined
> console.log(bool, typeof bool);
true 'boolean' VM1703:1
< undefined
> console.log(undef, typeof undef);
undefined 'undefined' VM1811:1
```

```

> console.log(nul, typeof nul);
null 'object' VM1905:1
< undefined
> console.log(bigInt, typeof bigInt);
5784678465846n 'bigint' VM2021:1
< undefined
> console.log(sym, typeof sym);
f Symbol() { [native code] } 'function' VM2119:1
< undefined

```

Task 3.2 —Object & Array: Run the code below, then modify it: add a 4th subject to the array and print it. Add an isPassing property to the student object and print it.

```

> const student = {
  name: "Ankit",
  rollNo: 2424113,
  course: "BIT"
};
< undefined
> console.log(student.name);
Ankit VM2450:1
< undefined
> const subjects = ["JavaScript", "DBMS", "Networking"];
< undefined
> console.log(subjects[0], subjects[2]);
JavaScript Networking VM2761:1
< undefined

```

Task 3.3 —The typeof null Trick: Predict typeof null before running it. Were you right? Write one sentence explaining why it behaves this way.

```

> let nul = null;
< undefined
> console.log(typeof nul);
object VM2968:1
< undefined

```

Part 4 — Operators & Expressions

Task 4.1 — Arithmetic: Pick two numbers and print their sum, difference, product, quotient, and remainder.

```

> console.log(2+5, 5-2, 2*5, 2/5);
7 3 10 0.4 VM3070:1
< undefined

```

Task 4.2 — == vs === Investigation: Run all six of the following and record true/false for each. Then explain in your own words why the == and === results differ for each pair.

Expression	Predicted	Actual	Reason
5 == "5"	true	true	Loose equality allows type coercion.
5 === "5"	false	false	Strict equality checks type as well as value.
0 == false	true	true	Loose equality coerces compatible values.
0 === false	false	false	Number and Boolean are different types.
null == undefined	true	true	Special loose-equality rule treats them as equal.
null === undefined	false	false	Strict equality does not coerce and the types differ.

```

> 5 == "5"
< true
> 5 === "5"
< false
> 0 == false
< true
> 0 === false
< false
> null == undefined
< true
> null === undefined
< false

```

Task 4.3 — Logical Operators: Run the code below, then change the values of age and hasID until the first line prints false. Record the values you used.

```

> let age = 20;
< undefined
> let hasID = true;
< undefined
> console.log(age>=18 && hasID);
true VM3463:1
< undefined
> console.log(age>=18 || hasID);
true VM3567:1
< undefined
> console.log(!hasID);
false VM3648:1
< undefined
> age = 16;
< 16
> console.log(age>=18 && hasID);
false VM3774:1
< undefined

```

Task 4.4 — Ternary Operator: Write a ternary expression that stores “Pass” in a variable if a marks variable is ≥ 40 , otherwise “Fail” Test it with two different values of marks.

```

> let marks = 80;
< undefined
> let result = marks >= 50 ? "pass" : "fail";
< undefined
> console.log(result)
pass VM3990:1
< undefined

```

Part 6 — Debugging Practice

Fix these 4 broken snippets. In your lab record, write the corrected code and note what error each one caused.

Snippet 1

Broken code:

```

> let total = 10 +
  console.log(total);
NaN VM26:2
< undefined

```

Corrected code:

```
> let total = 10 + 5;
  console.log(total);
15
VM30:2
< undefined
```

Reason: The + operator was incomplete because no value was provided after it.

Snippet 2

Broken code:

```
> const rate = 5;
  rate = 10;
✖ ▶ Uncaught TypeError: Assignment to constant variable.
   at <anonymous>:2:6
VM34:2
```

Corrected code:

```
> let rate = 5;
  rate = 10;
10
VM35:2
```

Reason: A variable declared with const cannot be reassigned.

Snippet 3

Broken code:

```
> console.log(username);
  let username = "admin";
✖ ▶ Uncaught ReferenceError: username is not defined
   at <anonymous>:1:13
VM315:1
```

Corrected code:

```
> let username = "admin";
  console.log(username);
admin
VM319:2
< undefined
```

Reason: A let variable cannot be accessed before its declaration.

Snippet 4

Broken code:

```
> let 2ndAttempt = "retry";
✖ Uncaught SyntaxError: Invalid or unexpected token (at VM323:1:5)
VM323:1
```

Corrected code:

```
> let secondAttempt = "retry";  
← undefined
```

Reason: A JavaScript variable name cannot start with a number.

Short Paragraph

The most confusing part of today's lab was understanding the difference between `==` and `===`. Practicing different examples and checking the results in the console helped me understand that `==` allows type conversion, while `===` checks both value and type.